Βιβλιογραφία

[1] C. Horstmann, Η γλώσσα προγραμματισμού Java, Αναλυτική Προσέγγιση. Broken Hill Publishers, 2021.

[2] J. Farrell, Java, Εκμάθηση με πρακτικά παραδείγματα. Κριτική, 2018.

Κεφάλαιο 16

Σύνοψη
Στην ενότητα αυτή παρουσιάζεται ο μηχανισμός των εξαιρέσεων. Καταρχάς εξηγείται η λειτουργία της στοίβας κλήσης, στη συνέχεια διαφοροποιούνται οι ελεγχόμενες από τις προαιρετικά ελεγχόμενες εξαιρέσεις και τα Errors. Μετά, παρουσιάζονται κατά σειρά, οι τεχνικές σύλληψης, η πρόταση finally, οι εξαιρέσεις που ορίζονται από τον χρήστη, ο μηχανισμός διαχείρισης και η σύνθετη σύλληψη. Τέλος, παρουσιάζονται οι βασικές προκαθορισμένες εξαιρέσεις της Java.

Προαπαιτούμενη γνώση
Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration. Οι διεπαφές στην Java. Η βασική διαχείριση των αρχείων.

Λέξεις κλειδιά
Στοίβα κλήσης (call stack), Εξαίρεση (exception), Error, ελεγχόμενη (checked) εξαίρεση, προαιρετικά ελεγχόμενη (unchecked) εξαίρεση.

16 Εξαιρέσεις

Μερικές φορές κατά την εκτέλεση μιας εφαρμογής είναι δυνατόν να συμβεί κάποιο απροσδόκητο γεγονός τέτοιο που η εφαρμογή δεν μπορεί να διαχειριστεί τουλάχιστον στο πλαίσιο της κανονικής ροής της. Για παράδειγμα, η εφαρμογή χρειάζεται να διαβάσει δεδομένα από αρχείο που όμως δεν βρίσκεται στην αναμενόμενη θέση. Η αντιμετώπιση τέτοιων απροσδόκητων συμβάντων γίνεται με την αξιοποίηση του μηχανισμού των εξαιρέσεων (exceptions).

Ας βάλουμε όμως τα θέματα σε μια σειρά.

16.1 Η στοίβα κλήσης

Έχουμε ήδη δει πώς μπορούμε να χρησιμοποιούμε τη λέξη throw για να παράγουμε μια εξαίρεση τύπου RuntimeException. Ας το δούμε λίγο πιο συγκεκριμένα με τη βοήθεια του κώδικα 16.1

```
public class CallStackUn {

    static int a() {
        b();
        return 0;
    }

    static int b() {
        c();
        return 0;
    }

    static void c() {
        throw new RuntimeException("exception occured in function c");
    }

    public static void main(String[] args) {
//openfiles
        a();
    }
}
```

Κώδικας 16.1 *Παραγωγή RuntimeException*

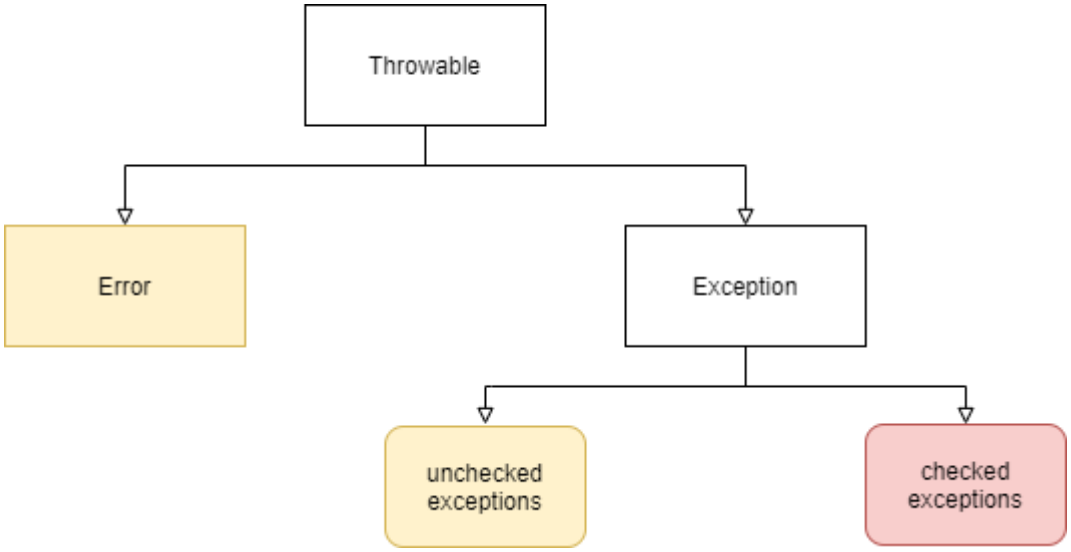
Στον κώδικα 16.1, η main καλεί την a, η a καλεί την b, η b την c και τέλος η c με χρήση της λέξης-κλειδί throw παράγει μια εξαίρεση τύπου RuntimeException [1]. Ενδιαφέρον έχει η έξοδος του κώδικα που έχει ως εξής:

```
Exception in thread "main" java.lang.RuntimeException:
exception occurred in function c
     at packagename.CallStackUn.c(CallStackUn.java:21)
     at packagename.CallStackUn.b(CallStackUn.java:16)
     at packagename.CallStackUn.a(CallStackUn.java:11)
     at packagename.CallStackUn.main(CallStackUn.java:25)
C:\Users\lmous\AppData\Local\NetBeans\Cache\12.4\executor-
snippets\run.xml:111: The following error occurred while
executing this line:
C:\Users\lmous\AppData\Local\NetBeans\Cache\12.4\executor-
snippets\run.xml:68: Java returned: 1
```

Η έξοδος αυτή μας πληροφορεί ότι εκδηλώθηκε μια εξαίρεση στην εφαρμογή μας και εμφανίζει τη στοίβα κλήσης (the call stack). Διαβάζοντας τη στοίβα κλήσης βλέπουμε πως η εξαίρεση προκλήθηκε στη γραμμή 21 μέσα στη συνάρτηση c που κλήθηκε από την β στη γραμμή 16 η οποία είχε κληθεί από την a στη γραμμή 11 που με τη σειρά της κλήθηκε από την main στην γραμμή 25 του κώδικα.

16.2 Τύποι απροσδόκητων λαθών

Η Java διαθέτει ένα μεγάλο σύνολο από προκαθορισμένους τύπους απροσδόκητων λαθών. Όπως θα δούμε στη συνέχεια αυτής της ενότητας υπάρχει η δυνατότητα, η εφαρμογή να ορίσει τους δικούς της τύπους. Όλες οι κλάσεις που αναπαριστούν τύπους λαθών, προκαθορισμένους ή οριζόμενους από τον χρήστη είναι παράγωγες της κλάσης Throwable.



Σχήμα 16.1 *Οργάνωση των τύπων απροσδόκητων λαθών*

Όπως φαίνεται στο σχήμα 16.1 οι τύποι των απροσδόκητων λαθών χωρίζονται καταρχάς σε δύο μεγάλες κατηγορίες, στα Errors και στα Exceptions. Τα δε Exceptions χωρίζονται περαιτέρω σε ελεγχόμενα (checked) και προαιρετικά ελεγχόμενα (unchecked). Ο μεταγλωττιστής υποχρεώνει τον προγραμματιστή να διαχειριστεί τις ελεγχόμενες εξαιρέσεις. Αντίθετα, η διαχείριση των προαιρετικών εξαιρέσεων δεν επιβάλλεται από τον μεταγλωττιστή. Παρόμοια, ο προγραμματιστής δεν υποχρεώνεται να διαχειριστεί λάθη τύπου Error ή κλάσεων παράγωγων της Error. Ας σημειωθεί πως στο σχήμα 16.1, το τμήμα Error αντιπροσωπεύει και όλη την ιεραρχία των παράγωγων κλάσεων της κλάσης Error. Παρόμοια, το τμήμα unchecked exceptions αντιπροσωπεύει την ιεραρχία των προαιρετικά ελεγχόμενων εξαιρέσεων και το τμήμα checked exceptions την ιεραρχία των ελεγχόμενων εξαιρέσεων.

Παραδείγματα κλάσεων παράγωγων της Error είναι οι κλάσεις IOError και η OutOfMemoryError. Λάθος τύπου IOError μπορεί να παραχθεί όταν ένα σοβαρό και μη αντιμετωπίσιμο πρόβλημα εκδηλώνεται στο σύστημα Εισόδου/Εξόδου. Το OutofMemoryError παράγεται στην περίπτωση που η εικονική μηχανή της Java εξαντλήσει τη διαθέσιμη μνήμη. Και στις δύο περιπτώσεις, η εφαρμογή δεν μπορεί να κάνει οτιδήποτε άλλο παρά να τερματίσει άμεσα τη λειτουργία της. Γενικότερα, Errors παράγονται από λάθη που οφείλονται σε παράγοντες εξωτερικούς στην εφαρμογή για τα οποία η εφαρμογή δεν μπορεί να υιοθετήσει αποτελεσματικό τρόπο αντιμετώπισης.

Παραδείγματα προαιρετικά ελεγχόμενων εξαιρέσεων συνιστούν οι κλάσεις NullPointerException, ArrayIndexOutOfBoundsException και IllegalArgumentException. Οι μη ελεγχόμενες αφορούν λάθη που οφείλονται στην εφαρμογή αλλά είναι τέτοιος ο χαρακτήρας τους που η εφαρμογή δεν μπορεί να κάνει κάτι παρά να τερματίσει. Για παράδειγμα, αν η εφαρμογή μας παράγει λάθος τύπου NullPointerException, τότε κάποιο λάθος (bug) έχει ο κώδικάς μας, οπότε αναγκαστικά θα διακοπεί η εκτέλεση της εφαρμογής και θα προσπαθήσουμε να εντοπίσουμε και να διορθώσουμε το λάθος. Μη ελεγχόμενες εξαιρέσεις είναι όλες οι παράγωγες κλάσεις της RuntimeException.

Χαρακτηριστικά παραδείγματα ελεγχόμενων εξαιρέσεων αποτελούν οι κλάσεις IOExeption και FileNotFoundException. FileNotFoundException παράγεται όταν η εφαρμογή προσπαθεί να προσπελάσει ένα αρχείο και αυτό δεν βρεθεί στη δεδομένη διαδρομή, ενώ IOException μπορεί να παραχθεί όταν για παράδειγμα επιχειρήσουμε να γράψουμε σε ένα αρχείο στο οποίο επιτρέπεται μόνο η ανάγνωση. Και στις δύο περιπτώσεις είναι δυνατόν η εφαρμογή να μπορεί να αποκριθεί. Για παράδειγμα, μπορεί το FileNotFoundException να προκλήθηκε από λανθασμένη επιλογή διαδρομής από τον χρήστη. Σε αυτήν την περίπτωση, μπορεί η εφαρμογή να βγάλει ένα μήνυμα προς τον χρήστη και να ζητήσει τη σωστή διαδρομή. Γενικότερα, οι ελεγχόμενες εξαιρέσεις αφορούν λάθη από τα οποία μια σωστά σχεδιασμένη εφαρμογή έχει τη δυνατότητα να ανακάμψει. Ελεγχόμενες εξαιρέσεις αναπαριστούν όλες οι κλάσεις που είναι παράγωγες της κλάσης Exception και δεν είναι παράγωγες της RuntimeError.

16.3 Σύλληψη εξαιρέσεων

Όπως αναφέραμε τις ελεγχόμενες εξαιρέσεις μας υποχρεώνει ο μεταγλωττιστής να τις διαχειριστούμε. Για μια ελεγχόμενη εξαίρεση που παράγεται μέσα σε μία συνάρτηση, έχουμε δύο δυνατότητες: Να τη συλλάβουμε για να τη διαχειριστούμε περαιτέρω ή απλώς να την προωθήσουμε. Στον κώδικα 16.2 προωθούμε την εξαίρεση τύπου Exception που παράγεται στη συνάρτηση c. Η προώθηση επιτυγχάνεται καταρχάς από την απουσία κώδικα που θα τη συλλάμβανε και από τη λέξη-κλειδί throws που ακολουθεί τη λίστα παραμέτρων της c συνοδευόμενη από το όνομα της εξαίρεσης που προωθείται.

```
public class CallStackCh {

    static int a() throws Exception {
        b();
        return 0;
    }

    static int b() throws Exception {
        c();
        return 0;
    }

    static void c() throws Exception {
        throw new Exception("exception occured in function c");
    }

    public static void main(String[] args) throws Exception {
        a();
    }
}
```

Κώδικας 16.2 *Αντιμετώπιση ελεγχόμενων εξαιρέσεων ως μη ελεγχόμενες.*

Το ερώτημα είναι τώρα τι σημαίνει προώθηση, δηλαδή προς τα πού προωθείται η εξαίρεση. Η εξαίρεση ακολουθεί όλη τη στοίβα κλήσης σε αντίστροφη κατεύθυνση είτε έως ότου κάποια συνάρτηση από τη στοίβα κλήσης συλλάβει την εξαίρεση ή έως τον τερματισμό της εφαρμογής και την εκτύπωση της στοίβας κλήσης. Έτσι στον κώδικα 16.2, η εξαίρεση μεταβιβάζεται στην b. Εκεί μπορεί να υπάρχει κώδικας που τη συλλαμβάνει ή απλώς μπορεί να προωθείται και πάλι. Στο παράδειγμά μας προωθείται. Πού; Στη συνάρτηση από την οποία κλήθηκε η b, δηλαδή στην a. Η δε a προωθεί και πάλι στην main και η main τερματίζει την εφαρμογή με τιμή επιστροφής 1 και προκαλεί εκτύπωση της στοίβας κλήσης. Αν τρέξετε τον κώδικα 16.2, θα διαπιστώσετε πως η έξοδός του είναι παρόμοια με αυτήν του κώδικα 16.1. Αυτό συμβαίνει, γιατί παρότι στον κώδικα 16.2 έχουμε ελεγχόμενη εξαίρεση δεν τη συλλαμβάνουμε αλλά την επαναπροωθούμε σε όλες τις συναρτήσεις της στοίβας κλήσης.

Ας δούμε τώρα πώς συλλαμβάνουμε μια εξαίρεση. Σε οποιαδήποτε συνάρτηση της στοίβας κλήσης μπορεί η εξαίρεση να συλληφθεί. Η σύλληψη της εξαίρεσης γίνεται με τη βοήθεια ενός μπλοκ try…catch. Αντικαταστήστε στον κώδικα 16.2 τη συνάρτηση a με τη συνάρτηση a που δίνεται στον κώδικα 16.3.

```
static int a() {
    try {
        b();
    } catch (Exception e) {
        System.out.println(
                e.getClass() + " " + e.getMessage() + " caught in a");
    }
    return 0;
}
```

Κώδικας 16.3 *Σύλληψη της εξαίρεσης στη συνάρτηση a.*

Η συνάρτηση a δεν προωθεί πλέον την εξαίρεση αλλά τη συλλαμβάνει. Έτσι η λέξη-κλειδί throws λείπει από την επικεφαλίδα της συνάρτησης, ενώ ο κώδικας από όπου μπορεί να παραχθεί εξαίρεση μέσα στην a, δηλαδή η συνάρτηση b περικλείεται μέσα σε try…catch μπλοκ. Μέσα στο try μπλοκ τοποθετείται ο κώδικας που μπορεί να παράγει εξαίρεση. Στη συνέχεια ακολουθεί το catch μπλοκ που μπορεί να συλλάβει την εξαίρεση εφόσον ελέγχει τον τύπο της. Όπως θα δούμε στη συνέχεια, ένα try μπλοκ μπορεί να ακολουθείται από πολλαπλά catch μπλοκ καθένα από τα οποία συλλαμβάνει διαφορετικές εξαιρέσεις. Τα catch μπλοκ είναι γνωστά ως διαχειριστές εξαιρέσεων (exception handlers). Στον κώδικα 16.3 ελέγχεται μία εξαίρεση τύπου Exception δηλαδή του τύπου που παρήχθη από την c, επομένως συλλαμβάνεται.

Τι είναι όμως αυτό που παράγεται στην c και στη συνέχεια προωθείται από συνάρτηση σε συνάρτηση ώσπου να συλληφθεί στην a; Είναι ένα αντικείμενο τύπου Exception. Στο catch μπλοκ που γίνεται η σύλληψη της εξαίρεσης το αντικείμενο αυτό είναι διαθέσιμο. Το μόνο που χρειάζεται να κάνει ο προγραμματιστής είναι να το ονομάσει ώστε να μπορεί να το χρησιμοποιήσει. Στον κώδικα 16.3, το αντικείμενο έχει ονομαστεί e. Στη συνέχεια, μέσα στην catch, χρησιμοποιώντας το αναγνωριστικό e αναφερόμαστε στο αντικείμενο τύπου Exception που παρήχθη στην c. Το αντικείμενο e μεταφέρει πληροφορίες που αφορούν το λάθος που συνέβη στην c.

Το μήνυμα που θα τυπωθεί τώρα είναι

```
class java.lang.Exception exception occured in function c
caught in a
```

Όπως είπαμε οποιαδήποτε συνάρτηση στη στοίβα κλήσης δύναται να συλλάβει την εξαίρεση εφόσον διαθέτει το κατάλληλο try…catch μπλοκ. Ας αντικαταστήσουμε τώρα και τη συνάρτηση b με τον κώδικα 16.4.

```
static int b() throws Exception {
    try {
        c();
    } catch (Exception e) {
        System.out.println(
                e.getClass() + " " + e.getMessage() + " caught in b");
    }
    c();
```

```
        return 0;
    }
```

Κώδικας 16.4 *Σύλληψη της εξαίρεσης στη συνάρτηση b.*

Το μήνυμα που λαμβάνουμε τώρα είναι

```
class java.lang.Exception exception occured in function c
caught in b
```

Αυτό είναι αναμενόμενο, καθώς η εξαίρεση προωθείται πρώτα στην b και μετά στην a. Από τη στιγμή που η b συλλαμβάνει την εξαίρεση, η προώθησή της καταστέλλεται. Παρομοίως, θα μπορούσε να συλληφθεί η εξαίρεση στη συνάρτηση c όπως δείχνει ο κώδικας 16.5:

```
    static void c() throws Exception {
        try {
            throw new Exception("exception occured in function c");
        } catch (Exception e) {
            System.out.println(
                    e.getClass() + " " + e.getMessage() + " caught in c");
        }
    }
```

Κώδικας 16.5 *Σύλληψη της εξαίρεσης στη συνάρτηση c όπου παρήχθη αρχικά.*

Από όσα παραθέσαμε μέχρι τώρα φαίνεται πως ο μηχανισμός των εξαιρέσεων μας δίνει τη δυνατότητα να συνδυάσουμε πληροφορίες που είναι γνωστές στον κώδικα που παράγει την εξαίρεση με πληροφορίες στον κώδικα που τη συλλαμβάνει. Με άλλα λόγια, παρέχεται η δυνατότητα να συνδυαστούν πληροφορίες που αφορούν το λάθος που προκάλεσε την εξαίρεση με πληροφορίες διαθέσιμες στο περιβάλλον κλήσης του κώδικα που συμβαίνει το λάθος. Αυτή η δυνατότητα είναι συχνά αναγκαία για την ορθή αντιμετώπιση ενός απροσδόκητου λάθους.

16.4 Η πρόταση finally

Ας δούμε τώρα μια εφαρμογή διαχείρισης αρχείων στην οποία αξιοποιείται η διαχείριση εξαιρέσεων:

```
static void copyStudents(String source, String target) throws IOException {
    File input = new File(source);
    BufferedWriter out = null;
    Scanner scanner = null;
    int[] k = new int[2];

    try {
        scanner = new Scanner(input);
        FileWriter fW = new FileWriter(target);
        out = new BufferedWriter(fW);
        while (scanner.hasNextLine()) {
            String st = scanner.nextLine();
            out.write(st + "\n");
            k[2] = 1;
        }
    } catch (FileNotFoundException ex) {
        System.out.println(ex + " file " + source + " has not copied");
    } catch (IOException ex) {
        System.out.println(ex + " file " + source + " has not copied");
    }

    if (scanner != null) {
        scanner.close();
    }
```

```
        if (out != null) {
            out.close();
        }
    }
```

Κώδικας 16.6 *Αντιγραφή αρχείων με σύλληψη εξαιρέσεων*

Ο κώδικας 16.6 αντιγράφει το αρχείο που βρίσκεται στη θέση source στο αρχείο που προσδιορίζεται από τη θέση target. Καταρχάς, οι διαχειριστές των δύο αρχείων, δηλαδή οι μεταβλητές scanner και out αρχικοποιούνται στο null. Η αρχικοποίηση αυτή είναι απαραίτητη. Μέσα στην try επιχειρείται η σύνδεση των διαχειριστών των αρχείων με τα φυσικά αρχεία στον δίσκο με αξιοποίηση των δημιουργών της Scanner, της FileWriter και της BufferedWriter. Και οι τρεις δημιουργοί ενδέχεται να αποτύχουν είτε παράγοντας IOException είτε παράγοντας FileNotFoundException. Αν υποθέσουμε ότι μέσα στην try παράγεται IOException ή FileNotFoundException, ο κώδικας θα μεταβεί στην αντίστοιχη catch η οποία στην ουσία παράγει ένα μήνυμα και στη συνέχεια θα συνεχίσει η ροή προς το τελευταίο τμήμα της συνάρτησης όπου κλείνουμε τα αρχεία. Αν το λάθος συμβεί στον δημιουργό της Scanner, η μεταβλητή scanner ενδέχεται να μην έχει λάβει τιμή, η δε μεταβλητή out σίγουρα δεν θα έχει λάβει τιμή. Για να αποφύγουμε να καλέσουμε την close μέσω μιας αναρχικοποίητης μεταβλητής, αρχικοποιούμε τις scanner και out σε null, οπότε πριν τις χρησιμοποιήσουμε για κλείσιμο των αρχείων ελέγχουμε αν είναι null. Αν η τιμή κάποιου διαχειριστή είναι όντως null, σημαίνει πως το αντίστοιχο αρχείο δεν άνοιξε και επομένως δεν είναι αναγκαίο να το κλείσουμε. Εξάλλου η κλήση οποιασδήποτε μη στατικής συνάρτησης μέσω του null δεν είναι παρά ένα επιπλέον λάθος στον κώδικά μας. Αν είχαμε αφήσει τις μεταβλητές αναρχικοποίητες, ο έλεγχος αυτός θα ήταν αδύνατος. Προσέξτε πως δεν κλείνουμε τον FileWriter, καθώς αυτός αποτελεί υποκείμενη δομή του BufferedWriter το κλείσιμο του οποίου προκαλεί αυτόματα κλείσιμο και του FileWriter.

Μία άλλη σημαντική παρατήρηση είναι η εξής: Οι δύο εξαιρέσεις που αναφέραμε προηγουμένως συνδέονται μεταξύ τους με σχέσεις κληρονομικότητας. Πιο συγκεκριμένα, η FileNotFoundException είναι παράγωγη της IOException. Με άλλα λόγια, μια FileNotFoundException είναι μία IOException. Επομένως αν στο catch μπλοκ τοποθετήσουμε πρώτα την IOException και μετά την FileNotFoundException, τότε στην περίπτωση που θα παραχθεί FileNotFoundException αυτή θα συλληφθεί από τον διαχειριστή της IOException. Ευτυχώς, ο μεταγλωττιστής της Java εξετάζει τη σειρά που συλλαμβάνουμε τις συγγενικές εξαιρέσεις και δεν επιτρέπει τέτοιου είδους λάθη.

Προσέξτε πως στην επικεφαλίδα της copyStudent έχει προστεθεί το τμήμα throws IOException. Αυτό οφείλεται στο ότι IOException μπορεί να παραχθεί και από τις κλήσεις close στο τέλος του κώδικα. Στην περίπτωση που έχουμε εξαίρεση εκεί δεν μας είναι ιδιαίτερα χρήσιμη η διαχείρισή της, γιατί πολύ απλά δεν μπορούμε να κάνουμε κάτι για ένα τέτοιο σφάλμα. Από τη στιγμή που δεν διαχειριζόμαστε μια πιθανή εξαίρεση είμαστε υποχρεωμένοι να την προωθήσουμε στη στοίβα κλήσης.

Επίσης, η scanner.hasNextLine μπορεί να παράγει εξαίρεση τύπου IllegalStateException αν το αρχείο είναι κλειστό, η δε scanner.nextLine μπορεί να παράγει επίσης IllegalStateException ή NoSuchElementException στην περίπτωση που δεν υπάρχει άλλη γραμμή στο αρχείο. Οι εξαιρέσεις αυτές είναι και οι δύο προαιρετικά ελεγχόμενες. Θα μπορούσαμε βεβαίως να προσθέσουμε κώδικα ώστε να τις συλλάβουμε. Ωστόσο, στον συγκεκριμένο κώδικα αποκλείονται αυτές οι εξαιρέσεις, καθώς το αρχείο έχει ήδη ανοίξει αμέσως πριν κληθεί η hasNextLine και η nextLine έχει κληθεί αφού έχει προηγηθεί έλεγχος με την hasNextLine.

Παρόλα όσα έχουμε κάνει, αν τρέξουμε τον κώδικα 16.6 θα διαπιστώσουμε πως αυτός "πέφτει" εγείροντας μία εξαίρεση ArrayIndexOutOfBoundsException. Αυτό οφείλεται στην πρόταση k[2]=1. Ο πίνακας k έχει μήκος 2 και επομένως μόνο οι θέσεις του 0 και 1 είναι διαθέσιμες. Η απόπειρα να διαβάσουμε στη θέση 2 παράγει την εξαίρεση ArrayIndexOutOfBoundsException. Αυτή είναι προαιρετικά ελεγχόμενη. Θα μπορούσαμε να προσθέσουμε ένα catch για αυτήν την εξαίρεση. Ωστόσο πρόκειται για εξαίρεση που οφείλεται σε προγραμματιστικό λάθος. Μία τέτοια εξαίρεση μπορεί να παραχθεί από οπουδήποτε και δεν είναι καλή πρακτική να βάζουμε όλο τον κώδικα σε try μπλοκ για να συλλάβουμε μία τέτοια εξαίρεση ή άλλες μη ελεγχόμενες εξαιρέσεις. Επιπλέον, δεν μπορούμε να προσφέρουμε κάποια βοήθεια στον χρήστη κατά τον χρόνο εκτέλεσης. Το μόνο που μπορούμε να κάνουμε είναι να τερματίσουμε την εφαρμογή και να διορθώσουμε τον κώδικα, πράγμα για το οποίο δεν χρειάζεται σύλληψη της εξαίρεσης. Εξάλλου μετά την απολαθοποίηση του κώδικα τέτοια εξαίρεση δεν θα παράγεται.

Εκείνο όμως που αποτελεί πρόβλημα είναι πως η ArrayIndexOutOfBoundsException δεν επιτρέπει την ομαλή ροή του κώδικα και άρα την εκτέλεση των close. Το αποτέλεσμα είναι ότι τα αρχεία παραμένουν ανοιχτά

και είναι πιθανή η απώλεια δεδομένων. Χρειαζόμαστε λοιπόν έναν μηχανισμό ο οποίος θα εγγυάται το κλείσιμο των αρχείων είτε παράγεται εξαίρεση οποιουδήποτε τύπου είτε όχι. Αυτόν ακριβώς τον μηχανισμό προσφέρει η Java με τη δεσμευμένη λέξη finally [2]. Όπως δείχνει ο κώδικας 16.7, ένα μπλοκ finally ακολουθεί τα μπλοκ catch. Μέσα στο μπλοκ finally τοποθετούμε κώδικα που θέλουμε να εκτελεστεί έτσι κι αλλιώς, είτε παραχθεί εξαίρεση είτε όχι.

```
    static void copyStudentsF(String source, String target) throws IOException {
        File input = new File(source);
        BufferedWriter out = null;
        Scanner scanner = null;
        int[] k = new int[2];

        try {
            scanner = new Scanner(input);
            FileWriter fW = new FileWriter(target);
            out = new BufferedWriter(fW);
            while (scanner.hasNextLine()) {
                String st = scanner.nextLine();
                out.write(st + "\n");
                k[2] = 1;
            }
        } catch (FileNotFoundException ex) {
            System.out.println(ex + " file " + source + " has not copied");
        } catch (IOException ex) {
            System.out.println(ex + " file " + source + " has not copied");
        } finally {
            if (scanner != null) {
                scanner.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
```

Κώδικας 16.7 *Σύλληψη εξαιρέσεων με μπλοκ finally*

Ένα θέμα που προκύπτει από τη χρήση του finally μπλοκ είναι πως στην περίπτωση που παραχθεί εξαίρεση τόσο μέσα στο try μπλοκ όσο και μέσα στο finally μπλοκ, τότε η συνάρτηση στην οποία περικλείεται το try…catch…finally θα προωθήσει την εξαίρεση που παράχθηκε στο finally και όχι αυτή που παράχθηκε στο try. Αυτή που παράγεται στο try καταστέλλεται. Το θέμα αυτό μπορεί να αντιμετωπιστεί με τη δομή try-with-resources που προστέθηκε στην έκδοση 7.

```
    static void copyStudentsWR(String source, String target) throws IOException {
        File input = new File(source);

        try ( BufferedWriter out = new BufferedWriter(new FileWriter(target));
Scanner scanner = new Scanner(input);) {
            while (scanner.hasNextLine()) {
                String st = scanner.nextLine();
                out.write(st + "\n");
            }
        } catch (FileNotFoundException ex) {
            System.out.println(ex + " file " + source + " has not copied");
        } catch (IOException ex) {
            System.out.println(ex + " file " + source + " has not copied");
        }
    }
```

Κώδικας 16.8 *try with resources*

Στον κώδικα 16.8 χρησιμοποιούμε την try-with-resources [3]. Εκείνο που περιλαμβάνει επιπλέον της απλής try είναι οι παρενθέσεις μεταξύ της δεσμευμένης λέξης try και του μπλοκ που ακολουθεί την try. Μέσα σε αυτές τις παρενθέσεις δημιουργούμε τα απαιτούμενα αντικείμενα μέσω των οποίων θα διαχειριστούμε πόρους, όπως τα αρχεία στο παράδειγμά μας. Οποιοδήποτε αντικείμενο ανήκει σε κλάση που υλοποιεί τη διεπαφή AutoCloseable μπορεί να δημιουργηθεί μέσα στις παρενθέσεις. Στη συνέχεια, όλοι οι δεσμευμένοι πόροι θα κλείσουν αυτόματα. Με την try-with-resources σε περίπτωση που εκδηλωθεί εξαίρεση κατά το κλείσιμο των πόρων αλλά και μέσα στην try, η κύρια συνάρτηση θα προωθήσει την εξαίρεση που εκδηλώθηκε στην try.

16.5 Εξαιρέσεις οριζόμενες από τον χρήστη

Αν μεταβείτε στην τεκμηρίωση της κλάσης Exception, θα διαπιστώσετε πως αυτή διαθέτει δεκάδες παράγωγες κλάσεις. Μάλιστα, κάθε μία διαθέτει αρκετά περιγραφικό και μακρύ όνομα. Γενικά, στις βιβλιοθήκες της Java έχει επικρατήσει η τάση για κάθε διαφορετικό τύπο λάθους να ορίζεται μία αντίστοιχη κλάση εξαιρέσεων. Έτσι στις περισσότερες περιπτώσεις ο τύπος και μόνο της παραγόμενης εξαίρεσης μας δίνει αρκετές πληροφορίες για το λάθος που την προκάλεσε.

Είτε θελήσουμε να ακολουθήσουμε στις εφαρμογές μας αυτήν τη σχεδιαστική προσέγγιση είτε όχι, συχνά θα προκύψει η ανάγκη να ορίσουμε νέους τύπους εξαιρέσεων. Για παράδειγμα, έστω ότι διαβάζουμε τα στοιχεία των σπουδαστών από οριοθετημένο αρχείο και όλες οι λειτουργίες επιτελούνται σωστά. Καμία εξαίρεση τύπου IOException ή FileNotFoundException δεν παράγεται και το αρχείο διαβάζεται γραμμή προς γραμμή με επιτυχία. Σε κάθε γραμμή όμως του αρχείου περιμένουμε ανάμεσα σε άλλα και 5 βαθμούς που διαχωρίζονται μεταξύ τους με ένα διάστημα, π.χ. 10.0 11.0 12.0 13.0 14.0 15.0. Αντί όμως της αναμενόμενης τιμής σε μια γραμμή του αρχείου, έχουμε 2 βαθμούς που δεν διαχωρίζονται μεταξύ τους, π.χ. 10.0 11.0 12.0 13.014.0 15.0. Σε αυτήν την περίπτωση, μπορούμε να χρησιμοποιήσουμε μια προκαθορισμένη γενική εξαίρεση, όπως η Exception ή να ορίσουμε δικό μας τύπο εξαίρεσης.

Για να ορίσουμε μια προαιρετικά ελεγχόμενη θα πρέπει να δημιουργήσουμε μια κλάση που είναι παράγωγη της RuntimeException ή παράγωγη κάποιας κλάσης απογόνου της RuntimeException. Για να ορίσουμε ελεγχόμενη εξαίρεση θα πρέπει να δημιουργήσουμε μία κλάση παράγωγη της Exception ή κάποιου απογόνου της Exception εξαιρουμένης της RuntimeException και των απογόνων της.

```
public class MallFormedFileLine extends Exception {
    private final int line;

    public MallFormedFileLine(int line) {
        this.line = line;
    }

    public int getLine() {
        return line;
    }

    @Override
    public String toString() {
        return "MallFormedFileLine{" + "line=" + line + '}';
    }
}
```

Κώδικας 16.9 *Παράδειγμα εξαίρεσης οριζόμενης από τον χρήστη*

Στον κώδικα 16.9 η κλάση MallFormedFileLine είναι παράγωγη της κλάσης Exception. Η MallFormedFileLine σχεδιάστηκε έτσι ώστε να μπορεί να χρησιμοποιηθεί σε παραλλαγή της readStudents του κώδικα 15.3 όπου σε ένα οριοθετημένο αρχείο κάποια γραμμή δεδομένων ενδέχεται να μην είναι διαμορφωμένη όπως αναμένεται.

```
static ArrayList<Student> readStudents(String fName) {
    ArrayList<Student> rVal = new ArrayList<>();
    File input = new File(fName);
    int ln = 0;
    try ( Scanner scanner = new Scanner(input);) {
        while (scanner.hasNextLine()) {
```

```
                ln++;
                String st = scanner.nextLine();
                String[] stFields = st.split(" ");
                if (stFields.length != 10) {
                    throw new MallFormedFileLine(ln);
                }
                Student student = create(stFields);
                rVal.add(student);
            }
        } catch (FileNotFoundException ex) {
            System.out.println(ex + ": " + fName);
        } catch (MallFormedFileLine e) {
            System.out.println("Format Error at line: " + e.getLine());
        }
        return rVal;
    }
```

Κώδικας 16.10 *Αξιοποίηση της MallFormedFileLine*

Κάθε γραμμή του γνωστού μας αρχείου studentsd.dat αναμένεται να διαχωρίζεται σε 10 τμήματα. Όταν αυτό δεν συμβαίνει, τότε έχουμε κάποιο λάθος στη γραμμογράφηση του αρχείου. Σε αυτήν την περίπτωση μπορούμε να παράγουμε μια MallFormedFileLine όπως φαίνεται στον κώδικα 16.10. Η MallFormedFileLine μας πληροφορεί σε ποια ακριβώς γραμμή του αρχείου υπάρχει το πρόβλημα.

16.6 Διαχείριση και σύνθετη σύλληψη

Στον κώδικα 16.10 με τη σύλληψη της MallFormedFileLine διακόπηκε η φόρτωση των στοιχείων των σπουδαστών στο ArrayList. Ως αποτέλεσμα, το ArrayList τα στοιχεία του οποίου εμφανίζονται στην main περιέχει μόνο τα στοιχεία των σπουδαστών που προηγούνται της γραμμής που προκάλεσε την εξαίρεση. Ανάλογα όμως με τη φύση της εφαρμογής είναι δυνατόν η φόρτωση του αρχείου να χρειάζεται να συνεχιστεί με τις υπόλοιπες γραμμές του αρχείου εφόσον έχουν σωστή γραμμογράφηση.
Ένα άλλο θέμα που δεν αντιμετωπίζει ο κώδικας 16.10 είναι η εξαίρεση NumberFormatException. Πρόκειται για μη ελεγχόμενη εξαίρεση που ενδέχεται να παραχθεί από κάποια κλήση Double.parseDouble μέσα στην create. Τέτοια εξαίρεση θα παραχθεί όταν ένα String δεν μπορεί να μετατραπεί σε double. Για παράδειγμα, μια γραμμή του αρχείου έχει την ακόλουθη μορφή:

```
212 Jack Green 11.0 12.0 13.0 14.0 15.0 16.0 2005-05-01
```

Οι δύο εξαιρέσεις, η MallFormedFileLine και η NumberFormatException αναφέρονται σε παρόμοια περίπου λάθη και ενδεχομένως να θέλουμε να χρησιμοποιήσουμε τον ίδιο διαχειριστή και για τις δύο.
Στον κώδικα 16.11 η νέα έκδοση της readStudents δίνει απαντήσεις σε αυτά τα ζητήματα:

```
    static ArrayList<Student> readStudentsC(String fName) {
        ArrayList<Student> rVal = new ArrayList<>();
        File input = new File(fName);
        Scanner scanner = null;
        Scanner keyboard=new Scanner(System.in);
        try {
            scanner = new Scanner(input);
        } catch (FileNotFoundException ex) {
            System.out.println(ex + ": " + fName);
            System.exit(1);
        }
        int ln = 0;
        while (true) {
            ln++;
            try {
                if (!scanner.hasNextLine())
                    break;
```

```
                String st = scanner.nextLine();
                String[] stFields = st.split(" ");
                if (stFields.length != 10) {
                    throw new MallFormedFileLine(ln);
                }
                Student student = create(stFields);
                rVal.add(student);

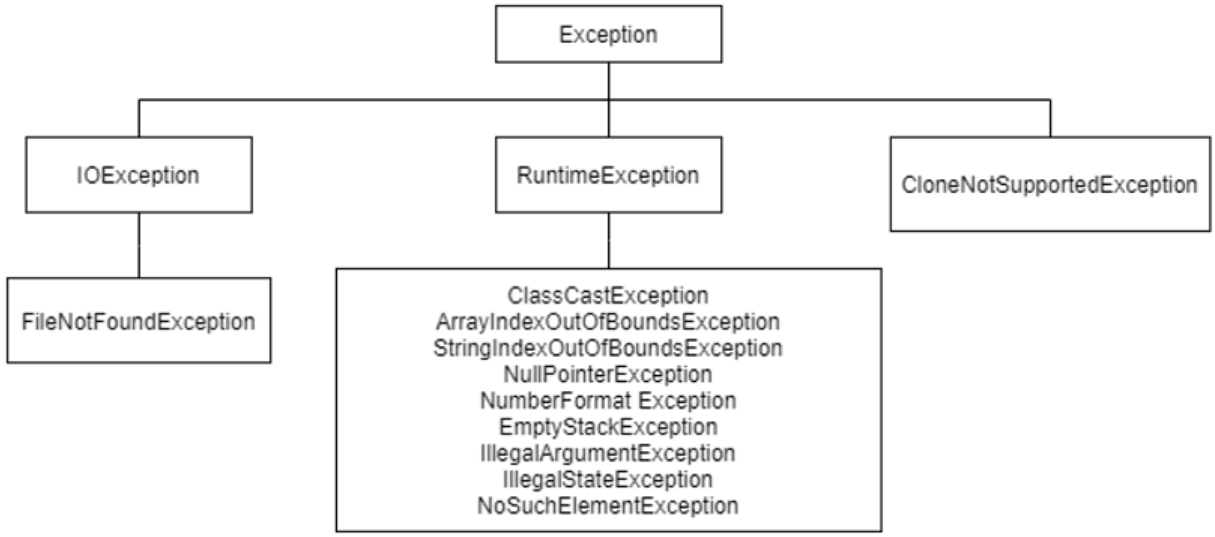
            } catch (MallFormedFileLine | NumberFormatException e) {
                System.out.println(e);
                System.out.println("Continue reading Y/N? ");
                String userAnswer = keyboard.nextLine();
                if (userAnswer.equals("N")) {
                    scanner.close();
                    System.exit(1);
                }
            }
        }
        scanner.close();
        return rVal;
    }
```

Κώδικας 16.11 *Διαχείριση και σύνθετη σύλληψη*

Στον κώδικα 16.11 καταρχάς συνδέουμε ένα αντικείμενο Scanner με τη ροή τυπικής εξόδου. Η ροή αυτή είναι μονίμως ανοικτή, εφόσον δεν την κλείσει η εφαρμογή μας και έτσι η σύνδεση της Scanner δεν κινδυνεύει να αποτύχει, οπότε την πραγματοποιούμε έξω από την try. Στη συνέχεια μέσα στην try ανοίγουμε το αρχείο εισόδου, δηλαδή το αρχείο που είναι αποθηκευμένα τα στοιχεία των μαθητών. Αν το άνοιγμα του αρχείου αποτύχει, τότε απλώς βγάζουμε ένα μήνυμα και καλούμε την System.exit με παράμετρο 1. Η χρήση της System.exit θέλει προσοχή, καθώς προκαλεί άμεσο τερματισμό της εικονικής μηχανής και της εφαρμογής μας. Επομένως, μετά την κλήση της δεν είναι δυνατόν να γίνει οριστικοποίηση πόρων. Στο συγκεκριμένο παράδειγμα όμως εφόσον το αρχείο δεν βρέθηκε, δεν χρειάζεται και να κλείσει. Οπότε αν η εφαρμογή συνεχίσει, είμαστε βέβαιοι πως η σύνδεση μεταξύ αρχείου και scanner έχει επιτευχθεί. Στη συνέχεια, τοποθετούμε την try μέσα σε μια επαναληπτική διαδικασία η οποία διακόπτεται όταν τελειώσουν οι γραμμές του αρχείου. Αν παραχθεί εξαίρεση, έχουμε έναν διαχειριστή και για τους δύο τύπους που πιθανώς θα παραχθούν. Αυτό επιτυγχάνεται με την κάθετο που χωρίζει τους τύπους των εξαιρέσεων και είναι γνωστό ως piping. Στην περίπτωση που εφαρμόζουμε piping, ας έχουμε υπόψη πως το αντικείμενο e είναι final, ενώ αν δεν εφαρμόζεται piping δεν είναι. Στη συνέχεια εμφανίζουμε το αντικείμενο που παρέχει ικανοποιητική πληροφόρηση σε σχέση με το λάθος και ρωτάμε τον χρήστη αν επιθυμεί να συνεχίσει η φόρτωση του αρχείου ή όχι. Αν απαντήσει αρνητικά, κλείνουμε το αρχείο και καλούμε την System.exit(1), διαφορετικά εκτελείται το επόμενο βήμα επανάληψης που επιχειρεί να διαβάσει την επόμενη γραμμή του αρχείου.

16.7 Βασικές προκαθορισμένες εξαιρέσεις

Η ιεραρχία των εξαιρέσεων της Java περιλαμβάνει πολλές δεκάδες υποκλάσεις. Σε αυτήν την ενότητα παραθέτουμε ένα υποσύνολο που περιλαμβάνει εξαιρέσεις που χρησιμοποιούνται συχνά, πάντα στο πλαίσιο της μέχρι εδώ διδασκόμενης ύλης.



Σχήμα 16.2 *Ιεραρχία βασικών προκαθορισμένων εξαιρέσεων*

Η κλάση Exception στην κορυφή της ιεραρχίας είναι η γονική κλάση όλων των εξαιρέσεων στην Java. Συνήθως, όταν θέλουμε να ορίσουμε δικό μας τύπο εξαίρεσης, αυτήν την κλάση κληρονομούμε.

Η κλάση IOException είναι η γενική εξαίρεση που δείχνει πως κάποιο είδος προβλήματος έχει συμβεί κατά την είσοδο ή έξοδο δεδομένων στην εφαρμογή.

Εξαίρεση τύπου FileNotFoundException παράγεται από τους δημιουργούς των κλάσεων FileInputStream, FileOutputStream, and RandomAccessFile όταν το αρχείο με το οποίο προσπαθούν να συνδεθούν δεν βρίσκεται στην προκαθορισμένη διαδρομή ή κάποια άλλη αιτία δεν επιτρέπει το άνοιγμά του.

CloneNotSupportedException παράγεται όταν επιχειρήσουμε να επανορίσουμε τη συνάρτηση clone σε μία κλάση που δεν υλοποιεί τη διεπαφή Cloneable.

Οι υπόλοιπες εξαιρέσεις του σχήματος 16.2 είναι μη ελεγχόμενες.

Η RuntimeException είναι η γονική κλάση όλων των μη ελεγχόμενων εξαιρέσεων της Java.

ClassCastException παράγεται όταν επιχειρηθεί η μετατροπή του τύπου ενός αντικειμένου σε μη συμβατή κλάση.

ArrayIndexOutOfBoundsException παράγεται όταν η εφαρμογή επιχειρήσει να διαβάσει ή να γράψει έξω από τα όρια ενός πίνακα.

StringIndexOutOfBoundsException παράγεται όταν η εφαρμογή επιχειρήσει να διαβάσει χαρακτήρα ή άλλο τμήμα ενός String αλλά υπερβεί τα όρια του String.

NullPointerException παράγεται όταν διαμέσου μεταβλητής αναφοράς που έχει την τιμή null επιχειρηθεί η κλήση μεθόδου ή η προσπέλαση μεταβλητής στιγμιότυπου. Λάβετε όμως υπόψη ότι δια μέσου μεταβλητής αναφοράς με τιμή null είναι επιτρεπτή η κλήση στατικών μελών χωρίς πρόβλημα αρκεί βέβαια να πρόκειται για μέλη της κλάσης στην οποία αναφέρεται η μεταβλητή.

NumberFormatException παράγεται όταν επιχειρηθεί η μετατροπή ενός String σε αριθμό αλλά το String δεν είναι μετατρέψιμο, π.χ. “120!”.

EmptyStackException παράγεται όταν επιχειρηθεί pop από άδειο Stack.

IllegalArgumentException παράγεται όταν σε συνάρτηση περνάμε λανθασμένα πραγματικές παραμέτρους. Για παράδειγμα, στον κώδικα 7.14, η factorial παράγει RuntimeException όταν κληθεί με αρνητική παράμετρο. Θα μπορούσε η RuntimeException να αντικατασταθεί με IllegalArgumentException που είναι πιο εξειδικευμένη σε σχέση με το συγκεκριμένο λάθος.

IllegalStateException παράγεται όταν επιχειρούμε ενέργειες σε αντικείμενο Scanner το οποίο είναι κλειστό και σε άλλες αντίστοιχες περιπτώσεις.

NoSuchElementException παράγεται όταν επιχειρήσουμε να διαβάσουμε δεδομένα που δεν υπάρχουν. Για παράδειγμα, αν καλέσουμε την next ενός Iterator χωρίς να έχουμε ελέγξει με την hasNext ή αν καλέσουμε την nextLine ενός Scanner χωρίς να έχουμε ελέγξει με την hasNextLine είναι πιθανή η παραγωγή της.